

人工智能课程内容总结

——期末复习要点整理——

基于教材《人工智能应用实践教程（Python 实现）》

目录

1 第一部分：人工智能概述	5
1.1 人工智能的定义与发展历史	5
1.1.1 人工智能发展的三次浪潮	5
1.2 人工智能的三大学派	5
1.3 强人工智能与弱人工智能	5
1.4 机器学习与深度学习概述	6
1.4.1 机器学习的类型	6
1.4.2 深度学习	6
1.5 人工智能主要应用领域	6
2 第二部分：Python 开发环境基础	6
2.1 Python 语言特点	6
2.2 NumPy 基础	6
2.3 Scikit-learn 简介	7
2.4 Matplotlib 绘图基础	7
3 第三部分：核心机器学习算法	7
3.1 线性回归（Linear Regression）	7
3.1.1 基本模型	7
3.1.2 损失函数——均方误差（MSE）	8
3.1.3 R^2 ——决定系数（拟合优度）	8
3.1.4 梯度下降法	9
3.1.5 Python 实现示例	9
3.1.6 梯度下降的手动实现	10
3.2 逻辑斯蒂分类（Logistic Regression）	10
3.2.1 Sigmoid 函数	10
3.2.2 损失函数——交叉熵（Cross-Entropy）	11
3.2.3 分类评价指标	11
3.2.4 Python 实现示例	12
3.3 K-近邻分类与 K-均值聚类	13
3.3.1 K-近邻（KNN）原理	13
3.3.2 距离度量	13
3.3.3 Python 实现示例	13
3.3.4 K-均值聚类（K-Means）	14
3.4 朴素贝叶斯分类（Naive Bayes）	15
3.4.1 贝叶斯定理	15

3.4.2	朴素条件独立假设	15
3.4.3	离散型特征朴素贝叶斯	15
3.4.4	连续型特征朴素贝叶斯	16
3.4.5	两种模型的对比	16
3.4.6	Python 实现——离散型朴素贝叶斯 (MultinomialNB)	16
3.4.7	拉普拉斯平滑效果演示	17
3.4.8	Python 实现——连续型朴素贝叶斯 (GaussianNB)	18
3.4.9	两种模型的选择指南	20
3.5	模型评估与选择	20
3.5.1	训练集/验证集/测试集划分	20
3.5.2	交叉验证 (Cross-Validation)	20
3.5.3	欠拟合与过拟合——系统化处理	21
3.5.4	其他经典机器学习算法概述	23
4	第四部分：神经网络与深度学习	24
4.1	从线性模型到神经网络	24
4.1.1	为什么需要多层？	24
4.2	单神经元的前向计算——手算示例	25
4.3	全连接神经网络结构	25
4.3.1	前向传播——矩阵形式	25
4.4	激活函数——为什么必须是非线性的？	26
4.5	欠拟合与过拟合——模型容量选择	26
4.6	TensorFlow 2.0 与 Keras 简介	27
4.7	损失函数——沿用已有知识	28
4.8	反向传播——链式法则求梯度	28
4.9	训练流程总结——与单层模型的对比	28
4.10	Python 实现——Keras 全连接网络 (MNIST 手写数字识别)	29
4.11	从零实现——NumPy 版全连接网络	30
4.12	卷积神经网络 (CNN) 基础——卷积、ReLU 与池化	31
4.12.1	卷积层 (Convolution Layer)	31
4.12.2	ReLU 激活层	31
4.12.3	池化层 (Pooling Layer)	32
4.12.4	Python 实现——卷积 + ReLU + 池化	32
5	第五部分：知识表示	34
5.1	谓词逻辑表示法	34
5.1.1	Python 实现——谓词逻辑归结推理演示	35
5.2	产生式表示法	36

5.3	语义网络表示法	37
5.3.1	Python 实现——语义网络构建与查询	37
5.4	框架表示法	39
5.5	知识图谱	40
5.5.1	Python 实现——知识图谱构建与查询	41
6	第六部分：推理方法	43
6.1	产生式推理	43
6.1.1	产生式系统的三大组成部分	43
6.1.2	冲突消解策略（Conflict Resolution）	43
6.1.3	正向推理（数据驱动）	43
6.1.4	逆向推理（目标驱动）	43
6.1.5	逆向推理 Python 实现	44
6.1.6	正向推理 Python 实现	45
6.2	可信度推理（Certainty Factor）	46
6.2.1	可信度模型	46
6.2.2	CF 的计算公式	46
6.2.3	CF 推理完整算例	47
6.2.4	Python 实现可信度推理	47
7	第七部分：搜索策略	48
7.1	状态空间问题求解	48
7.1.1	经典示例——二阶梵塔问题	48
7.1.2	盲目搜索：广度优先与深度优先	49
7.1.3	Python 实现——BFS 求解八数码问题	49
7.2	启发式搜索——A* 算法	50
7.2.1	估价函数	50
7.2.2	A* 算法的 Python 实现	51
7.3	遗传算法（Genetic Algorithm）	52
7.3.1	基本原理	52
7.3.2	Python 实现	52
8	附录：课后补充练习题要点	54
8.1	关键公式汇总	55

1 第一部分：人工智能概述

1.1 人工智能的定义与发展历史

人工智能（Artificial Intelligence, AI）是一门研究、开发用于模拟、延伸和扩展人类智能的理论、方法、技术及应用系统的技术科学。AI 以人类行为标准进行猜想、假设、研究、实验而造就出来的机器智能。

1.1.1 人工智能发展的三次浪潮

1. **第一次高峰（1956–1974）**：1956 年达特茅斯会议标志着 AI 诞生。计算机被广泛应用于数学和自然语言领域，但因数据不足、算力有限，AI 进入低谷。
2. **第二次高峰（1980–1987）**：专家系统兴起，通过知识库和推理机模拟人类专家决策。受限于专用硬件成本，约 7 年后再次进入低谷。
3. **第三次高峰（2012 至今）**：深度学习突破。标志事件：2011 年 IBM Watson 战胜人类冠军；2015 年 ImageNet 超越人类；2016 年 AlphaGo 战胜人类围棋冠军。

1.2 人工智能的三大学派

- **符号主义（Symbolism）**：物理符号系统假设，路线：启发式算法 → 专家系统 → 知识工程 → 知识图谱。
- **连接主义（Connectionism）**：模拟人脑神经元，通过人工神经网络实现智能。深度学习使其成为当前主流。
- **行为主义（Actionism）**：基于「感知—行动」的行为智能模拟，代表方法为强化学习。

1.3 强人工智能与弱人工智能

- **强 AI**：具备与人类同等或超越人类的智慧，具有心智和意识。目前是否可实现存在争议。
- **弱 AI**：专注于特定任务领域（CV、NLP、语音等）。当前所有 AI 应用均属弱 AI。
- **测试方法**：图灵测试、咖啡测试、机器人学生测试。

1.4 机器学习与深度学习概述

1.4.1 机器学习的类型

- **监督学习**：需要标注数据，学习输入 $\rightarrow$ 输出映射。包括回归（连续值）和分类（离散值）。
- **无监督学习**：无需标注，发现数据内在结构，如聚类、降维。
- **半监督学习**：少量标注 + 大量无标注数据结合学习。
- **迁移学习**：将已有模型调整后应用于新领域。

1.4.2 深度学习

深度学习核心是使用多层神经网络自动学习层次化特征。关系：人工智能 $\supset$ 机器学习 $\supset$ 深度学习。

1.5 人工智能主要应用领域

计算机视觉、自然语言处理、语音识别、推荐系统、自动驾驶、医疗诊断、金融风控、智能制造等。

2 第二部分：Python 开发环境基础

2.1 Python 语言特点

Python 是 AI 领域最主流的编程语言，具有简洁、易读、生态丰富的特点：

- **核心数据类型**：int, float, str, list, tuple, dict, set
- **列表操作**：切片、append/remove、列表推导式 `[x*2 for x in range(5)]`
- **字典操作**：keys/values/items、get 方法
- **循环结构**：for-in 循环、while 循环、条件判断

2.2 NumPy 基础

NumPy 是 Python 科学计算的基础库，提供高效的数组操作：

Listing 1: NumPy 数组基本操作

```
1 import numpy as np
2
3 # 创建数组
```

```

4 a = np.array([1, 2, 3, 4, 5])           # 一维
5 b = np.zeros((3, 4))                   # 全零矩阵
6 c = np.ones((2, 3))                   # 全一矩阵
7 d = np.random.randn(10)               # 正态分布随机数
8
9 # 数组操作
10 print(a.shape, a.dtype)               # 形状和类型
11 print(a.reshape(5, 1))                # 改变形状
12 print(np.dot(a, a))                  # 点积
13 print(b.sum(axis=0))                  # 按列求和

```

2.3 Scikit-learn 简介

Scikit-learn 是 Python 机器学习标准库，提供统一的 API：

- `fit(X, y)` — 训练模型
- `predict(X)` — 预测
- `score(X, y)` — 评估模型

2.4 Matplotlib 绘图基础

Listing 2: Matplotlib 基本绘图

```

1 import matplotlib.pyplot as plt
2
3 plt.figure(figsize=(8, 5))
4 plt.plot([1, 2, 3, 4], [1, 4, 9, 16], 'ro-', label='y=x^2')
5 plt.xlabel('x'); plt.ylabel('y')
6 plt.title('Matplotlib 示例'); plt.legend(); plt.grid(True)
7 plt.show()

```

3 第三部分：核心机器学习算法

3.1 线性回归 (Linear Regression)

3.1.1 基本模型

给定有 d 个属性的样本 $\mathbf{x} = (x_1, x_2, \dots, x_d)$ ，线性回归学习一个预测函数：

$$f(\mathbf{x}) = w_1x_1 + w_2x_2 + \cdots + w_dx_d + b = \mathbf{w}^T \mathbf{x} + b \quad (1)$$

其中 $\mathbf{w}$ 为权重向量, b 为偏置项。当 $d = 1$ 时为单变量线性回归: $f(x) = w_1x + b$ 。

3.1.2 损失函数——均方误差 (MSE)

$$J(\mathbf{w}, b) = \frac{1}{2m} \sum_{i=1}^m (f(\mathbf{x}^{(i)}) - y^{(i)})^2 \quad (2)$$

其中 m 为样本数量, 系数 $\frac{1}{2}$ 是为了求导时消去平方项的 2。

3.1.3 R^2 ——决定系数 (拟合优度)

评价回归模型好坏的另一个核心指标是 R^2 (决定系数, Coefficient of Determination), 它衡量模型对数据方差的解释程度:

$$R^2 = 1 - \frac{SS_{\text{res}}}{SS_{\text{tot}}} = 1 - \frac{\sum_{i=1}^m (y^{(i)} - \hat{y}^{(i)})^2}{\sum_{i=1}^m (y^{(i)} - \bar{y})^2} \quad (3)$$

其中 $\hat{y}^{(i)}$ 为模型预测值, $\bar{y} = \frac{1}{m} \sum y^{(i)}$ 为样本均值。

R^2 的含义与评价作用:

- **取值范围:** $R^2 \in (-\infty, 1]$, 越接近 1 说明模型拟合越好。当 $R^2 = 1$ 时, 模型完美预测所有样本; 当 $R^2 = 0$ 时, 模型与直接取均值的效果相同; $R^2 < 0$ 表示模型比取均值还差。
- **直观意义:** R^2 表示“因变量的变异中有百分之多少能被自变量解释”。例如 $R^2 = 0.85$ 表示自变量能解释因变量 85% 的变异。
- **与 MSE 的关系:** MSE 衡量绝对误差大小 (量纲与 y^2 一致), R^2 是归一化指标 (无量纲), 更适合不同数据集之间的比较。
- **局限性:** R^2 随特征数增加而单调不减, 增加无关特征也会使 R^2 上升。因此引入调整 R^2 (Adjusted R^2): $R_{\text{adj}}^2 = 1 - \frac{(1-R^2)(m-1)}{m-d-1}$, 其中 d 为特征数, 它会对无用特征施加惩罚。

Python 计算 R^2 示例:

Listing 3: 计算 R^2 得分

```
1 from sklearn.metrics import r2_score
2 from sklearn.linear_model import LinearRegression
3 import numpy as np
4
```



```

5 X = np.array([[75], [87], [105], [110], [120]])
6 y = np.array([270, 280, 295, 310, 335])
7
8 model = LinearRegression().fit(X, y)
9 y_pred = model.predict(X)
10
11 # R^2 计算
12 r2 = r2_score(y, y_pred)
13 print(f"R^2 = {r2:.4f}")    # 输出如 R^2 = 0.9812
14
15 # 手动验证: R^2 = 1 - SSE/SST
16 SSE = np.sum((y - y_pred) ** 2)
17 SST = np.sum((y - y.mean()) ** 2)
18 r2_manual = 1 - SSE / SST
19 print(f"手动 R^2 = {r2_manual:.4f}")    # 应与 sklearn 结果一致

```

3.1.4 梯度下降法

$$w_j := w_j - \alpha \frac{\partial J}{\partial w_j}, \quad b := b - \alpha \frac{\partial J}{\partial b} \quad (4)$$

其中 α 为学习率。对 MSE 求偏导：

$$\frac{\partial J}{\partial w_j} = \frac{1}{m} \sum_{i=1}^m (f(\mathbf{x}^{(i)}) - y^{(i)}) x_j^{(i)}$$

3.1.5 Python 实现示例

Listing 4: 线性回归——基于 Scikit-learn

```

1 import numpy as np
2 from sklearn.linear_model import LinearRegression
3 import matplotlib.pyplot as plt
4
5 # 训练数据：房屋面积(m^2) vs 总价(万元)
6 X = np.array([[75], [87], [105], [110], [120]])
7 y = np.array([270, 280, 295, 310, 335])
8
9 model = LinearRegression()
10 model.fit(X, y)
11
12 print(f"w1 = {model.coef_[0]:.4f}")

```

```

13 print(f"b = {model.intercept_:.4f}")
14 print(f"R^2 = {model.score(X, y):.4f}")
15
16 # 预测
17 new_area = np.array([[100], [130]])
18 pred = model.predict(new_area)
19 print(f"100m^2预测价: {pred[0]:.2f}万元")
20 print(f"130m^2预测价: {pred[1]:.2f}万元")
21
22 # 可视化
23 plt.scatter(X, y, color='blue', label='训练数据')
24 plt.plot(X, model.predict(X), 'r-', label='拟合直线')
25 plt.xlabel('房屋面积 (m^2)'); plt.ylabel('总价 (万元)')
26 plt.legend(); plt.grid(True); plt.show()

```

3.1.6 梯度下降的手动实现

Listing 5: 梯度下降手动实现单变量线性回归

```

1 def gradient_descent(X, y, alpha=0.01, epochs=1000):
2     m = len(y)
3     w, b = 0.0, 0.0
4     for _ in range(epochs):
5         y_pred = w * X.flatten() + b
6         dw = (1/m) * np.sum((y_pred - y) * X.flatten())
7         db = (1/m) * np.sum(y_pred - y)
8         w -= alpha * dw
9         b -= alpha * db
10    return w, b
11
12 w_gd, b_gd = gradient_descent(X, y, alpha=0.0001, epochs=10000)
13 print(f"梯度下降结果: w={w_gd:.4f}, b={b_gd:.4f}")

```

3.2 逻辑斯蒂分类 (Logistic Regression)

3.2.1 Sigmoid 函数

$$\sigma(z) = \frac{1}{1 + e^{-z}}, \quad z = \mathbf{w}^T \mathbf{x} + b \quad (5)$$

二分类中，正类 ($y = 1$) 和负类 ($y = 0$) 的后验概率为：

$$P(y = 1 | \mathbf{x}) = \frac{1}{1 + e^{-(\mathbf{w}^T \mathbf{x} + b)}}, \quad P(y = 0 | \mathbf{x}) = \frac{e^{-(\mathbf{w}^T \mathbf{x} + b)}}{1 + e^{-(\mathbf{w}^T \mathbf{x} + b)}} \quad (6)$$

3.2.2 损失函数——交叉熵（Cross-Entropy）

$$J(\mathbf{w}, b) = -\frac{1}{m} \sum_{i=1}^m \left[y^{(i)} \ln \hat{y}^{(i)} + (1 - y^{(i)}) \ln(1 - \hat{y}^{(i)}) \right] \quad (7)$$

3.2.3 分类评价指标

对于二分类问题，混淆矩阵（Confusion Matrix）定义如下：

- **TP（True Positive）**：真实为正类，预测也为正类——预测正确
- **TN（True Negative）**：真实为负类，预测也为负类——预测正确
- **FP（False Positive）**：真实为负类，预测为正类——误报（Type I 错误）
- **FN（False Negative）**：真实为正类，预测为负类——漏报（Type II 错误）

基于混淆矩阵的四个核心评价指标：

$$\text{准确率 Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \quad (8)$$

$$\text{精确率 Precision} = \frac{TP}{TP + FP} \quad (9)$$

$$\text{召回率 Recall} = \frac{TP}{TP + FN} \quad (10)$$

$$F_1 \text{ 分数} = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \quad (11)$$

各指标的意义与使用场景：

- **准确率（Accuracy）**：所有样本中预测正确的比例。适用于类别均衡的数据集。当正负样本严重不平衡时（如欺诈检测中 99.9% 为正常交易），即使全部预测为负类，准确率也能达到 99.9%，但模型毫无价值——准确率在不均衡数据上具有欺骗性。
- **精确率（Precision）**：预测为正类的样本中，真正为正类的比例。回答的问题是预测出的正类中有多少是靠谱的。适用于，“不能冤枉”的场景：垃圾邮件过滤（不希望正常邮件被误标）、搜索结果推荐（不希望用户看到无关内容）。
- **召回率（Recall）**：真实为正类的样本中，被正确找出的比例。回答的问题是真正的正类有多少被我们抓住了。适用于，“不能漏掉”的场景：疾病筛查（不希望漏掉任何一个患者）、逃税识别、安全漏洞检测。

- F_1 分数：精确率和召回率的调和平均数。调和平均（而非算术平均）的特点是对较小值更敏感——只有当 Precision 和 Recall 都较高时 F_1 才高。适用于需要同时兼顾精确率和召回率的场景。

Precision 与 Recall 的权衡 (Precision-Recall Trade-off):

- 提高分类阈值 → 更保守 → Precision ↑、Recall ↓
- 降低分类阈值 → 更激进 → Precision ↓、Recall ↑
- 实际应用中选择哪个指标优先取决于业务需求：医疗诊断优先 Recall（不能漏诊），商品推荐优先 Precision（用户不想看到无关商品）。

指标选择快速参考：

场景	优先指标	原因
垃圾邮件过滤	Precision	正常邮件不能被误删
疾病筛查	Recall	患者不能被漏诊
通用分类评估	F_1	平衡 Precision 与 Recall
类别均衡数据集	Accuracy	简单直观、信息充分
类别不均衡	F_1 或 AUC	Accuracy 具有欺骗性

3.2.4 Python 实现示例

Listing 6: 逻辑斯蒂分类——基于 Scikit-learn

```

1 import numpy as np
2 from sklearn.linear_model import LogisticRegression
3 from sklearn.metrics import accuracy_score, classification_report
4
5 X = np.array
6     ([[3.36, 78], [2.70, 75], [2.90, 80], [3.12, 100], [2.80, 125],
7      [3.32, 94], [3.05, 120], [3.70, 160], [3.52, 170], [3.60, 155]])
8
9 y = np.array([1, 1, 1, 1, 1, 0, 0, 0, 0, 0])
10
11 model = LogisticRegression(solver='lbfgs')
12 model.fit(X, y)
13
14 print(f"w = {model.coef_}, b = {model.intercept_}")

```

```

14 new_X = np.array([[3.00, 100], [3.50, 150]])
15 pred = model.predict(new_X)
16 prob = model.predict_proba(new_X)
17 for i, (p, pr) in enumerate(zip(pred, prob)):
18     status = "好卖" if p == 1 else "不好卖"
19     print(f"样本{i+1}: {status}, 概率: 不好卖={pr[0]:.3f}, 好卖={pr[1]:.3f}")
20
21 print(f"\n训练集准确率: {model.score(X, y):.2%}")
22 print(classification_report(y, model.predict(X), target_names=['不好卖', '好卖']))

```

3.3 K-近邻分类与 K-均值聚类

3.3.1 K-近邻 (KNN) 原理

给定测试样本，在训练集中找到与其最相似的 K 个邻居，由这些邻居的多数类别决定测试样本的类别。

3.3.2 距离度量

$$\text{欧氏距离 } L_2: \quad d(\mathbf{x}, \mathbf{y}) = \sqrt{\sum_{i=1}^d (x_i - y_i)^2} \quad (12)$$

$$\text{曼哈顿距离 } L_1: \quad d(\mathbf{x}, \mathbf{y}) = \sum_{i=1}^d |x_i - y_i| \quad (13)$$

$$\text{闵可夫斯基距离 } L_p: \quad d(\mathbf{x}, \mathbf{y}) = \left(\sum_{i=1}^d |x_i - y_i|^p \right)^{1/p} \quad (14)$$

3.3.3 Python 实现示例

Listing 7: KNN 分类示例

```

1 from sklearn.neighbors import KNeighborsClassifier
2 from sklearn.model_selection import train_test_split
3 from sklearn.datasets import load_iris
4
5 iris = load_iris()
6 X_train, X_test, y_train, y_test = train_test_split(
7     iris.data, iris.target, test_size=0.3, random_state=42)

```

```

8
9 knn = KNeighborsClassifier(n_neighbors=3, metric='euclidean')
10 knn.fit(X_train, y_train)
11 print(f"KNN 准确率: {knn.score(X_test, y_test):.2%}")
12
13 for k in [1, 3, 5, 7, 10]:
14     model = KNeighborsClassifier(n_neighbors=k).fit(X_train,
15         y_train)
16     print(f"K={k:2d}, 准确率={model.score(X_test, y_test):.3f}")

```

3.3.4 K-均值聚类 (K-Means)

目标：将 n 个样本划分到 K 个簇，使簇内距离平方和最小化：

$$\min_{\mathbf{C}} \sum_{k=1}^K \sum_{\mathbf{x} \in C_k} \|\mathbf{x} - \boldsymbol{\mu}_k\|^2 \quad (15)$$

其中 $\boldsymbol{\mu}_k = \frac{1}{|C_k|} \sum_{\mathbf{x} \in C_k} \mathbf{x}$ 是簇 C_k 的质心。

Listing 8: K-Means 聚类示例

```

1 from sklearn.cluster import KMeans
2 from sklearn.datasets import make_blobs
3 import matplotlib.pyplot as plt
4
5 X, _ = make_blobs(n_samples=300, centers=4, cluster_std=0.6,
6     random_state=0)
7
8 kmeans = KMeans(n_clusters=4, random_state=0, n_init='auto')
9 y_pred = kmeans.fit_predict(X)
10
11 plt.scatter(X[:,0], X[:,1], c=y_pred, cmap='viridis', s=30)
12 plt.scatter(kmeans.cluster_centers_[0,0],
13     kmeans.cluster_centers_[0,1],
14     c='red', marker='X', s=200, label='质心')
15 plt.legend(); plt.title('K-Means 聚类结果'); plt.show()
16 print(f"惯性(Inertia): {kmeans.inertia_: .2f}")

```

3.4 朴素贝叶斯分类 (Naive Bayes)

3.4.1 贝叶斯定理

$$P(y = c_k | \mathbf{x}) = \frac{P(\mathbf{x} | y = c_k) \cdot P(y = c_k)}{P(\mathbf{x})} \quad (16)$$

3.4.2 朴素条件独立假设

假设各特征在给定类别下条件独立：

$$P(\mathbf{x} | y = c_k) = P(x_1, x_2, \dots, x_d | y = c_k) = \prod_{i=1}^d P(x_i | y = c_k) \quad (17)$$

决策规则为：

$$\hat{y} = \arg \max_{c_k} P(y = c_k) \prod_{i=1}^d P(x_i | y = c_k) \quad (18)$$

完整后验概率形式：

$$P(y = c_k | \mathbf{x}) = \frac{P(y = c_k) \prod_{i=1}^d P(x_i | y = c_k)}{\sum_{j=1}^K P(y = c_j) \prod_{i=1}^d P(x_i | y = c_j)} \quad (19)$$

3.4.3 离散型特征朴素贝叶斯

当特征为离散（类别型）取值时， $P(x_i | y = c_k)$ 直接通过频率计数估计：

$$P(x_i = v | y = c_k) = \frac{\text{count}(x_i = v, y = c_k)}{\text{count}(y = c_k)} \quad (20)$$

即：在类别 c_k 的样本中，特征 x_i 取值为 v 的比例。

拉普拉斯平滑 (Laplace Smoothing)

若某个特征值在训练集中从未出现在某类别中，则式 (20) 的分子为 0，导致整个后验概率变为 0——即使其他特征强烈支持该类别也会被“一票否决”。拉普拉斯平滑为每个计数加一个常数 λ （通常 $\lambda = 1$ ，也称为加一平滑）：

$$P(x_i = v | y = c_k) = \frac{\text{count}(x_i = v, y = c_k) + \lambda}{\text{count}(y = c_k) + \lambda \cdot |V_i|} \quad (21)$$

其中 $|V_i|$ 是特征 x_i 的可能取值个数。 $\lambda = 1$ 时为拉普拉斯平滑， $\lambda < 1$ 时称为 Lidstone 平滑。

平滑的作用与直观理解：

- $\lambda = 0$ ：无平滑，等同于最大似然估计，未出现即为 0 概率
- $\lambda = 1$ ：拉普拉斯平滑，假设每个特征值在每类中至少“伪出现”一次

- λ 越大，先验（均匀分布）的影响越强，模型越保守

3.4.4 连续型特征朴素贝叶斯

当特征为连续数值时，无法使用计数方法。通常假设每个特征在给定类别下服从正态（高斯）分布：

$$P(x_i | y = c_k) = \frac{1}{\sqrt{2\pi\sigma_{i,k}^2}} \exp\left(-\frac{(x_i - \mu_{i,k})^2}{2\sigma_{i,k}^2}\right) \quad (22)$$

其中 $\mu_{i,k}$ 和 $\sigma_{i,k}^2$ 分别是类别 c_k 中特征 x_i 的均值和方差，从训练数据中估计：

$$\mu_{i,k} = \frac{1}{N_k} \sum_{j:y^{(j)}=c_k} x_i^{(j)}, \quad \sigma_{i,k}^2 = \frac{1}{N_k} \sum_{j:y^{(j)}=c_k} (x_i^{(j)} - \mu_{i,k})^2 \quad (23)$$

3.4.5 两种模型的对比

表 1: 离散型 vs 连续型朴素贝叶斯对比

对比维度	离散型（MultinomialNB）	连续型（GaussianNB）
特征类型	类别型/计数型（如户型、楼层）	连续数值型（如身高、体重）
概率估计	频率计数 + 拉普拉斯平滑	高斯分布 PDF
核心参数	条件概率表、平滑参数 λ	各类别均值 $\mu_{i,k}$ 、方差 $\sigma_{i,k}^2$
零概率处理	拉普拉斯平滑（ $\lambda \geq 0$ ）	正态分布自然非零
适用场景	文本分类、离散属性预测	身高体重分类、传感器数据

3.4.6 Python 实现——离散型朴素贝叶斯（MultinomialNB）

Listing 9: 离散型朴素贝叶斯——MultinomialNB 与拉普拉斯平滑

```

1 import numpy as np
2 from sklearn.naive_bayes import MultinomialNB
3 from sklearn.preprocessing import OneHotEncoder
4 from sklearn.metrics import accuracy_score, classification_report
5
6 # ===== 房屋好卖预测案例（离散特征） =====
7 # 特征：[户型，居室数，楼层]，标签：1=好卖，0=不好卖
8 raw_features = [
9     ['大户型', 4, '高楼层'], ['中户型', 3, '高楼层'],
10    ['中户型', 3, '中楼层'], ['中户型', 3, '高楼层'],
11    ['小户型', 2, '高楼层'], ['中户型', 2, '高楼层'],

```



```

12     ['小户型', 2, '低楼层'], ['中户型', 2, '高楼层'],
13     ['小户型', 3, '低楼层'], ['大户型', 3, '低楼层'],
14 ]
15 raw_labels = [1, 1, 1, 1, 1, 0, 0, 0, 0, 0]
16
17 # One-Hot编码将离散特征转为数值
18 encoder = OneHotEncoder(sparse_output=False)
19 X = encoder.fit_transform(raw_features)
20 y = np.array(raw_labels)
21
22 # 测试样本：[中户型，2居室，高楼层]
23 test_raw = [['中户型', 2, '高楼层']]
24 test_X = encoder.transform(test_raw)
25
26 # ===== 不同平滑参数对比 =====
27 for alpha in [0.0, 0.5, 1.0, 2.0, 5.0]:
28     model = MultinomialNB(alpha=alpha)
29     model.fit(X, y)
30     pred = model.predict(test_X)
31     prob = model.predict_proba(test_X)
32     print(f"alpha={alpha:.1f}: 预测={pred[0]}, "
33           f"P(不好卖)={prob[0,0]:.4f}, P(好卖)={prob[0,1]:.4f}")
34
35 # ===== 最佳alpha下的详细评估 =====
36 best_model = MultinomialNB(alpha=1.0) # 拉普拉斯平滑(默认)
37 best_model.fit(X, y)
38 print(f"\n训练准确率: {best_model.score(X, y):.2%}")
39 print(f"各类别先验概率: {np.exp(best_model.class_log_prior_)}")
40 print(f"条件概率矩阵形状: {best_model.feature_log_prob_.shape}")

```

3.4.7 拉普拉斯平滑效果演示

Listing 10: 拉普拉斯平滑的数值演示

```

1 import numpy as np
2
3 # 模拟：10个样本，二分类，某特征有3种取值{A,B,C}
4 counts = {
5     'class_0': {'A': 3, 'B': 2, 'C': 0}, # C从未在class_0出现!
6     'class_1': {'A': 0, 'B': 1, 'C': 4},

```

```

7 }
8 total_0, total_1 = 5, 5
9 n_values = 3 # 特征取值数
10
11 # 无平滑 (alpha=0) - C在class_0概率为0!
12 p_c_given_0_no_smooth = counts['class_0']['C'] / total_0
13 print(f"无平滑 P(C|class_0) = {p_c_given_0_no_smooth:.2f}")
14 print("=> 零概率问题! 任何含C的样本都会被判为class_1")
15
16 # 拉普拉斯平滑 (alpha=1)
17 alpha = 1.0
18 p_c_given_0_smooth = (counts['class_0']['C'] + alpha) / (total_0
19     + alpha * n_values)
20 p_a_given_1_smooth = (counts['class_1']['A'] + alpha) / (total_1
21     + alpha * n_values)
22 print(f"\n拉普拉斯平滑(alpha=1):")
23 print(f" P(C|class_0) = (0+1)/(5+1*3) = {p_c_given_0_smooth:.4f}
24     ")
25 print(f" P(A|class_1) = (0+1)/(5+1*3) = {p_a_given_1_smooth:.4f}
26     ")
27 print("=> 零概率被消除, 所有特征值都有非零概率")
28
29 # 不同alpha对后验概率的影响
30 def posterior_with_alpha(alpha):
31     """计算含特征C的测试样本在class_0下的后验"""
32     p_c0 = 0.5 # 先验
33     p_c_given_0 = (0 + alpha) / (5 + alpha * 3)
34     return p_c0 * p_c_given_0
35
36 for a in [0.0, 0.1, 0.5, 1.0, 2.0, 10.0]:
37     print(f" alpha={a:4.1f}: P(class_0|C) proportional to {
38         posterior_with_alpha(a):.6f}")

```

3.4.8 Python 实现——连续型朴素贝叶斯 (GaussianNB)

Listing 11: 连续型朴素贝叶斯——GaussianNB 性别分类

```

1 import numpy as np
2 from sklearn.naive_bayes import GaussianNB
3 from sklearn.metrics import accuracy_score

```

```
4
5 # ===== 性别分类（身高、体重、脚尺寸） =====
6 trainX = np.array([
7     [183, 82.10, 30.48], [180, 86.02, 27.94],
8     [170, 77.15, 30.48], [180, 75.26, 25.40], # 男
9     [153, 45.00, 15.24], [165, 58.80, 17.78],
10    [168, 55.00, 22.86], [175, 68.00, 22.86], # 女
11 ])
12 trainY = np.array([0, 0, 0, 0, 1, 1, 1, 1]) # 0=男, 1=女
13
14 # 训练高斯朴素贝叶斯
15 gnb = GaussianNB()
16 gnb.fit(trainX, trainY)
17
18 print(f"各类别先验概率: {gnb.class_prior_}")
19 print(f"男性均值: {gnb.theta_[0]}")
20 print(f"女性均值: {gnb.theta_[1]}")
21 print(f"男性方差: {gnb.var_[0]}")
22 print(f"女性方差: {gnb.var_[1]}")
23
24 # 预测测试样本: 身高175, 体重70, 脚尺寸28
25 testX = np.array([[175, 70, 28]])
26 pred = gnb.predict(testX)
27 prob = gnb.predict_proba(testX)
28 gender = "男" if pred[0] == 0 else "女"
29 print(f"\n预测性别: {gender}")
30 print(f"P(男|特征)={prob[0,0]:.4f}, P(女|特征)={prob[0,1]:.4f}")
31
32 # ===== Iris数据集演示 =====
33 from sklearn.datasets import load_iris
34 from sklearn.model_selection import train_test_split
35
36 X, y = load_iris(return_X_y=True)
37 X_train, X_test, y_train, y_test = train_test_split(
38     X, y, test_size=0.3, random_state=0)
39
40 gnb2 = GaussianNB()
41 gnb2.fit(X_train, y_train)
42 print(f"\nIris测试准确率: {gnb2.score(X_test, y_test):.2%}")
43
```

```
44 # 预测概率
45 probs = gnb2.predict_proba(X_test[:3])
46 for i, p in enumerate(probs):
47     pred_class = gnb2.predict([X_test[i]])[0]
48     print(f"样本{i}: 预测类={pred_class}, 概率={p}")
```

3.4.9 两种模型的选择指南

- 离散特征（如文本词频、类别属性）：使用 MultinomialNB，注意调整平滑参数 λ 。文本分类中 $\lambda = 1$ （默认）通常效果良好。
- 连续特征（如身高、温度、价格）：使用 GaussianNB，前提是特征近似服从正态分布。若偏离严重，可考虑先对特征做变换（如取对数）。
- 混合特征：可将连续特征离散化后用 MultinomialNB，或分别建模后集成。

3.5 模型评估与选择

在完成模型训练后，需要科学地评估模型性能并选择合适的模型。

3.5.1 训练集/验证集/测试集划分

将数据集划分为三部分：

- 训练集（Training Set）：用于训练模型参数，通常占总数据的 60%~70%
- 验证集（Validation Set）：用于调参和模型选择（如选 KNN 的 K 值、选正则化系数），通常占 15%~20%
- 测试集（Test Set）：仅用于最终评估，在模型开发过程中不能被用于任何调参决策，通常占 15%~20%

若将测试集误用于调参，会导致数据泄露（Data Leakage）——测试集上的好成绩不代表模型真正泛化能力强。

3.5.2 交叉验证（Cross-Validation）

当数据集较小时，固定划分可能因数据分配不均衡导致评估偏差。 k 折交叉验证将数据分为 k 份，每次用 $k - 1$ 份训练、1 份验证，重复 k 次取平均：

$$\text{CV Score} = \frac{1}{k} \sum_{i=1}^k \text{Score}_i$$

常用 $k = 5$ 或 $k = 10$ 。 k 折交叉验证能更稳定地估计模型泛化性能，是模型选择（如对比不同算法）的黄金标准。

3.5.3 欠拟合与过拟合——系统化理解

一、欠拟合 (Underfitting)

- **定义：**模型容量太小，无法捕捉数据中的真实规律，对训练数据本身也拟合不佳。
- **原因：**模型过于简单（如用线性模型拟合非线性数据）；特征数量不足；正则化强度过大。
- **判断方式：**
 - 训练集误差高 + 验证集误差也高，且两者接近
 - 学习曲线：训练和验证损失都维持在高位，不随样本量增加而改善
- **解决方案：**增加模型复杂度（更多特征、更高阶多项式）；减少正则化强度；增加训练轮数；在神经网络中增加层数或神经元数。

二、过拟合 (Overfitting)

- **定义：**模型在训练数据上学到了噪声和偶然模式，而非真实的数据规律。导致模型对训练数据表现极好，但对新数据表现很差——泛化能力弱。
- **原因：**模型过于复杂（参数过多）；训练数据太少；训练时间过长；数据中存在大量噪声；特征过多而样本不足。
- **判断方式：**
 - 训练集误差很低 + 验证集误差明显偏高，且两者差距持续增大
 - 学习曲线：训练损失持续下降，验证损失先降后升（U 形）
 - 交叉验证：训练集和验证集的性能差距过大
- **解决方案：**增加训练数据量（数据增强）；正则化（L1/L2/Elastic Net）；早停（Early Stopping）；Dropout（神经网络中随机丢弃神经元）；减少特征数量（特征选择/降维）；集成学习（Bagging）。

三、偏差-方差权衡 (Bias-Variance Trade-off)

欠拟合和过拟合可以从**偏差 (Bias)**和**方差 (Variance)**的角度理解：

$$\text{期望泛化误差} = \underbrace{(\text{Bias})^2}_{\text{偏差的平方}} + \underbrace{\text{Variance}}_{\text{方差}} + \underbrace{\text{Irreducible Error}}_{\text{不可约误差}}$$

- **偏差 (Bias)：**模型预测的期望值与真实值之间的差距。高偏差 → 欠拟合（模型太简单，学不到真实规律）。

- **方差 (Variance):** 模型在不同训练集上学到的函数的变化程度。高方差 → 过拟合 (模型对训练数据的微小变化过于敏感)。
- **权衡关系:** 偏差和方差通常是矛盾的——降低偏差会增加方差，反之亦然。模型选择就是在偏差和方差之间找到最优平衡点。

表 2: 欠拟合与过拟合对比

特征	欠拟合 (Underfitting)	过拟合 (Overfitting)
训练误差	高	低
验证误差	高	高
误差差距	小	大
偏差	高	低
方差	低	高
模型复杂度	太低	太高
典型对策	增加复杂度	正则化/增加数据

四、正则化 (Regularization) ——抑制过拟合的核心技术

在损失函数中加入对权重大小的惩罚项，迫使模型偏好小权重：

- **L1 正则化 (Lasso):** $J_{L1} = J_{\text{原}} + \lambda \sum |w_i|$ ，倾向于产生稀疏解 (部分权重变为 0)，具有特征选择效果
- **L2 正则化 (Ridge):** $J_{L2} = J_{\text{原}} + \lambda \sum w_i^2$ ，倾向于让所有权重均匀地小，但不为 0
- **Elastic Net:** 结合 L1 和 L2: $J = J_{\text{原}} + \lambda_1 \sum |w_i| + \lambda_2 \sum w_i^2$

其中 λ (或 α , 在 sklearn 中) 控制正则化强度。 λ 越大，模型越简单，偏差增加但方差减少。

Python 演示——欠拟合 vs 过拟合 vs 合适拟合：

Listing 12: 欠拟合、过拟合与合适拟合的对比

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 from sklearn.preprocessing import PolynomialFeatures
4 from sklearn.linear_model import LinearRegression
5 from sklearn.metrics import mean_squared_error, r2_score
6
7 # 生成带噪声的二次曲线数据
8 np.random.seed(42)
9 X = np.linspace(-3, 3, 30).reshape(-1, 1)

```

```

10 y = X.ravel()**2 - 2*X.ravel() + 1 + np.random.normal(0, 2, 30)
11
12 X_test = np.linspace(-3, 3, 100).reshape(-1, 1)
13 y_true = X_test.ravel()**2 - 2*X_test.ravel() + 1
14
15 degrees = [1, 2, 10] # 1=欠拟合, 2=合适, 10=过拟合
16 titles = ['欠拟合 (d=1)', '合适拟合 (d=2)', '过拟合 (d=10)']
17 fig, axes = plt.subplots(1, 3, figsize=(15, 4))
18
19 for ax, d, title in zip(axes, degrees, titles):
20     poly = PolynomialFeatures(degree=d)
21     X_poly = poly.fit_transform(X)
22     model = LinearRegression().fit(X_poly, y)
23
24     X_test_poly = poly.transform(X_test)
25     y_pred = model.predict(X_test_poly)
26
27     train_r2 = r2_score(y, model.predict(X_poly))
28     test_mse = mean_squared_error(y_true, y_pred)
29
30     ax.scatter(X, y, c='blue', s=30, label='训练数据')
31     ax.plot(X_test, y_pred, 'r-', lw=2, label=f'模型 (R^2={train_r2:.2f})')
32     ax.plot(X_test, y_true, 'g--', lw=1, alpha=0.5, label='真实函数')
33     ax.set_title(f'{title}\n训练R^2={train_r2:.3f}, 测试MSE={test_mse:.1f}')
34     ax.legend(fontsize=8)
35     ax.set_xlabel('x')
36     ax.set_ylabel('y')
37
38 plt.tight_layout()
39 plt.show()

```

其中 λ (或 α , 在 sklearn 中) 控制正则化强度。 λ 越大, 模型越简单, 偏差增加但方差减少。

3.5.4 其他经典机器学习算法概述

除了本课程重点讲解的线性回归、逻辑斯蒂、KNN、K-Means 和朴素贝叶斯, 机器学习领域还有其他重要算法。以下做简要介绍以建立完整知识体系:

- **决策树 (Decision Tree)**: 通过递归地选择最优特征进行分裂构建树状结构。ID3 使用信息增益、C4.5 使用增益率、CART 使用 Gini 不纯度。优点是可解释性强，缺点是容易过拟合。随机森林 (Random Forest) 通过集成多棵决策树显著改善了过拟合问题。
- **支持向量机 (SVM)**: 寻找最大化分类间隔的超平面，通过核函数 (Kernel) 将数据映射到高维空间解决非线性问题。常用核函数：线性核、多项式核、RBF (高斯) 核。SVM 在小样本、高维数据上表现优异。
- **集成学习 (Ensemble Learning)**: 组合多个弱学习器构成强学习器。Bagging (如随机森林) 并行训练多个模型投票；Boosting (如 AdaBoost、XGBoost、LightGBM) 串行训练，每个模型关注前一模型的错误样本。
- **降维 (Dimensionality Reduction)**: PCA (主成分分析) 通过线性变换将高维数据投影到低维空间，保留最大方差方向。t-SNE 常用于高维数据的可视化。

这些算法与课程核心内容的关系：

监督学习： $\underbrace{\text{线性回归、逻辑斯蒂}}_{\text{线性模型}} \cup \underbrace{\text{KNN、决策树、SVM}}_{\text{非线性模型}} \cup \underbrace{\text{神经网络}}_{\text{深度学习}}$

4 第四部分：神经网络与深度学习

4.1 从线性模型到神经网络

回顾第三部分的线性回归 $f(\mathbf{x}) = \mathbf{w}^T \mathbf{x} + b$ 和逻辑斯蒂分类 $\sigma(\mathbf{w}^T \mathbf{x} + b)$ ，它们的共同点是：对输入做一次加权求和，然后直接输出或经一个激活函数输出。这可以看作一个最简单的「神经元」：

$$\text{输出} = \sigma(\underbrace{w_1 x_1 + w_2 x_2 + \cdots + w_d x_d + b}_{\text{加权求和 (与线性回归完全相同)}})$$

神经网络的思想是：把多个这样的神经元堆叠起来。第一层神经元的输出作为第二层神经元的输入，层层递进。这样就得到多层神经网络（也叫多层感知机，MLP）。

4.1.1 为什么需要多层？

单层模型（即线性回归/逻辑斯蒂）只能学习**线性决策边界**。例如逻辑斯蒂分类在二维特征空间中只能画一条直线分开两类。多层网络加上非线性激活函数后，可以拟合任意复杂的决策边界：

- **第一层**学习低级特征组合——如「高单价 + 大面积」组合
- **第二层**在低级特征基础上学习更抽象的模式——如「高端房产」特征

- 层层递进，最终能表达高度非线性的决策边界

4.2 单神经元的前向计算——手算示例

先把一个神经元的工作过程说清楚。对比第三部分的线性回归公式：

$$\underbrace{z = w_1x_1 + w_2x_2 + w_3x_3 + b}_{\text{与式 (1) 的 } f(\mathbf{x}) \text{ 完全相同}} \xrightarrow{\text{激活}} a = g(z)$$

具体数值示例：设输入 $\mathbf{x} = [2.0, -1.0, 3.0]$ ，权重 $\mathbf{w} = [0.5, -0.3, 0.8]$ ，偏置 $b = 0.1$ ：

1. 加权求和： $z = 0.5 \times 2.0 + (-0.3) \times (-1.0) + 0.8 \times 3.0 + 0.1 = 1.0 + 0.3 + 2.4 + 0.1 = 3.8$
2. 激活（以 ReLU 为例）： $a = \max(0, 3.8) = 3.8$

如果使用 Sigmoid 激活： $a = \frac{1}{1 + e^{-3.8}} \approx 0.978$ 。由此可见，一个神经元就是一次线性回归 + 一次非线性变换。

4.3 全连接神经网络结构

全连接（Fully Connected / Dense）指的是：第 l 层的每个神经元与第 $l + 1$ 层的所有神经元都相连，每条连线有一个权重。

典型三层结构（以 MNIST 手写数字识别为例理解维度变化）：

$$\underbrace{\mathbf{x}}_{784\text{维}} \xrightarrow{\mathbf{W}_1(128 \times 784)} \underbrace{\mathbf{h}}_{128\text{维}} \xrightarrow{\mathbf{W}_2(64 \times 128)} \underbrace{\mathbf{h}_2}_{64\text{维}} \xrightarrow{\mathbf{W}_3(10 \times 64)} \underbrace{\hat{\mathbf{y}}}_{10\text{维}}$$

关键：权重矩阵的维度 = (输出维度 × 输入维度)，矩阵乘法 $\mathbf{W}\mathbf{x}$ 将输入映射到输出空间。

4.3.1 前向传播——矩阵形式

对任意层数，前向传播逐层执行两步操作：

1. 线性变换： $\mathbf{z}^{(l)} = \mathbf{W}_l \mathbf{a}^{(l-1)} + \mathbf{b}_l$ （矩阵乘法 + 偏置，与线性回归同）
2. 非线性激活： $\mathbf{a}^{(l)} = g(\mathbf{z}^{(l)})$ （引入非线性）

以两层隐含层为例，完整的矩阵公式：

$$\mathbf{z}^{(1)} = \mathbf{W}_1 \mathbf{x} + \mathbf{b}_1, \quad \mathbf{a}^{(1)} = \text{ReLU}(\mathbf{z}^{(1)}) \quad (24)$$

$$\mathbf{z}^{(2)} = \mathbf{W}_2 \mathbf{a}^{(1)} + \mathbf{b}_2, \quad \mathbf{a}^{(2)} = \text{ReLU}(\mathbf{z}^{(2)}) \quad (25)$$

$$\mathbf{z}^{(3)} = \mathbf{W}_3 \mathbf{a}^{(2)} + \mathbf{b}_3, \quad \hat{\mathbf{y}} = \text{Softmax}(\mathbf{z}^{(3)}) \quad (26)$$

与后面 NumPy 代码中的 `np.dot(X, self.W1) + self.b1` 完全对应。

4.4 激活函数——为什么必须是非线性的？

如果 $g(z) = z$ （恒等函数），则多层网络退化为单层： $\mathbf{W}_2(\mathbf{W}_1\mathbf{x}) = (\mathbf{W}_2\mathbf{W}_1)\mathbf{x}$ ，相当于一个 $\mathbf{W}'\mathbf{x}$ ，与线性回归无异。因此必须使用非线性激活函数。

三种最常用的激活函数及选择指南：

$$\text{Sigmoid: } \sigma(z) = \frac{1}{1 + e^{-z}} \in (0, 1) \quad (\text{二分类输出层，与逻辑斯蒂的式 (5) 完全相同}) \quad (27)$$

$$\text{ReLU: } \text{ReLU}(z) = \max(0, z) \quad (\text{隐含层首选——计算简单、缓解梯度消失}) \quad (28)$$

$$\text{Softmax: } S_i = \frac{e^{z_i}}{\sum_j e^{z_j}} \quad (\text{多分类输出层，输出各类别概率}) \quad (29)$$

记忆口诀：隐含层一律 ReLU，二分类输出 Sigmoid，多分类输出 Softmax。

4.5 欠拟合与过拟合——模型容量选择

关于欠拟合与过拟合的系统化定义、原因、判断方式和偏差-方差分析，请参见第 3.5.3 节。本节聚焦于神经网络场景下的具体表现和应对策略。

神经网络的容量（参数数量）决定了其可学习的函数空间大小：

- **欠拟合 (Underfitting)**：模型容量太小，无法捕捉数据中的真实规律。**特征**：训练集和测试集损失都很高，两者接近。**NN 特有方案**：增加层数（加深网络）；增加每层神经元数（加宽网络）；使用更复杂的架构（如从全连接换为 CNN/RNN）；减小正则化强度。
- **过拟合 (Overfitting)**：模型容量过大，不仅学到了数据规律还记住了噪声。**特征**：训练集损失很低但验证集损失很高，两者差距持续拉大。**NN 特有方案**：Dropout——训练时随机丢弃一定比例（如 50%）的神经元，强制网络学习冗余表示；早停 (Early Stopping)——监控验证集损失，当验证损失不再下降时停止训练；数据增强 (Data Augmentation)——对图像做旋转、翻转、裁剪等变换扩充训练集；批量归一化 (Batch Normalization)——对每层输入做归一化，兼具轻微正则化效果。

学习曲线判断法 (Learning Curves)：

- 训练损失和验证损失都维持高位且接近 → 欠拟合
- 训练损失持续下降、验证损失先降后升 (U 形) → 过拟合
- 训练和验证损失都降至低位且接近 → 合适拟合

4.6 TensorFlow 2.0 与 Keras 简介

TensorFlow 是由 Google 在 2015 年发布的开源深度学习框架，底端由 C++ 实现。TensorFlow 2.0 将 Keras 作为官方高阶 API，常用模块包括：

- `tf.data`: 输入数据管道，支持批量加载、数据增强
- `tf.keras`: 高阶神经网络 API，包含层、模型、优化器等
- `tf.image`: 图像处理（裁剪、变换、编码解码）
- `tf.losses`: 损失函数模块
- `tf.train`: 优化器、学习率衰减策略等训练组件

Keras 提供两种搭建模型的方式：

- **Sequential 模型**：层按顺序堆叠，适用于简单的线性拓扑网络
- **Functional API（函数式模型）**：支持多输入/多输出、残差连接、共享层等复杂拓扑

Keras Sequential 模型三步流程：

1. **构建**: `model = Sequential([Dense(64,activation=relu), Dense(10,activation=softmax)])`
2. **编译**: `model.compile(optimizer=adam, loss=categorical_crossentropy, metrics=[accuracy])`
3. **训练 + 评估**: `model.fit(x,y,epochs=10) → model.evaluate(x_test,y_test)`

Keras Functional API 示例：

Listing 13: Keras 函数式 API——多输入模型示例

```
1 from tensorflow.keras.layers import Input, Dense, concatenate
2 from tensorflow.keras.models import Model
3
4 # 两个独立的输入分支
5 input_a = Input(shape=(64,), name='input_a')
6 input_b = Input(shape=(32,), name='input_b')
7
8 x1 = Dense(32, activation='relu')(input_a)
9 x2 = Dense(16, activation='relu')(input_b)
10
```

```

11 merged = concatenate([x1, x2])
12 output = Dense(10, activation='softmax')(merged)
13
14 model = Model(inputs=[input_a, input_b], outputs=output)
15 model.compile(optimizer='adam', loss='categorical_crossentropy')
16 model.summary()

```

4.7 损失函数——沿用已有知识

神经网络的损失函数与第三部分的定义完全一致：

- 回归问题 → MSE（式2）： $L = \frac{1}{m} \sum (\hat{y}_i - y_i)^2$
- 二分类 → 交叉熵（式7）： $L = -\frac{1}{m} \sum [y_i \ln \hat{y}_i + (1 - y_i) \ln(1 - \hat{y}_i)]$
- 多分类 → 分类交叉熵： $L = -\frac{1}{m} \sum_i \sum_k y_{i,k} \ln \hat{y}_{i,k}$

4.8 反向传播——链式法则求梯度

直觉理解（与梯度下降对比）：

- 线性回归的梯度下降（式4）：损失 J 对 w 直接求偏导，然后 $w := w - \alpha \frac{\partial J}{\partial w}$
- 神经网络的反向传播：损失 L 对每一层的 $\mathbf{W}_l, \mathbf{b}_l$ 求偏导。由于网络有多层，需用链式法则从输出层逐层向回传递梯度

两者的参数更新公式完全相同： $\mathbf{W}^{(l)} := \mathbf{W}^{(l)} - \alpha \frac{\partial L}{\partial \mathbf{W}^{(l)}}$ 。

区别仅在于 $\frac{\partial L}{\partial \mathbf{W}^{(l)}}$ 的计算需要链式法则：

$$\frac{\partial L}{\partial \mathbf{W}^{(l)}} = \frac{\partial L}{\partial \hat{\mathbf{y}}} \cdot \frac{\partial \hat{\mathbf{y}}}{\partial \mathbf{z}^{(L)}} \cdot \frac{\partial \mathbf{z}^{(L)}}{\partial \mathbf{a}^{(L-1)}} \cdots \frac{\partial \mathbf{a}^{(l)}}{\partial \mathbf{z}^{(l)}} \cdot \frac{\partial \mathbf{z}^{(l)}}{\partial \mathbf{W}^{(l)}} \quad (30)$$

关键理解：链上每一项都是简单函数的导数——Sigmoid 导数 = $\sigma(1 - \sigma)$ ，ReLU 导数 = {0 或 1}，矩阵乘法对矩阵的导数 = 转置后的另一个矩阵。实际编程中框架（如 Keras/TensorFlow）自动完成这些计算。

4.9 训练流程总结——与单层模型的对比

一句话总结：神经网络 = 多层线性模型 + 非线性激活 + 反向传播。如果你理解了第三部分的梯度下降，神经网络的训练原理就已经掌握了——只是链更长、参数更多而已。

表 3: 线性回归 vs 神经网络的训练流程对比

步骤	线性回归/逻辑斯蒂	神经网络
模型形式	$f(\mathbf{x}) = \sigma(\mathbf{w}^T \mathbf{x} + b)$	多层 σ 嵌套
参数数量	$d + 1$ 个	数百至数百万
前向计算	一次点积	多层矩阵乘法 + 激活
损失函数	MSE / 交叉熵	相同
梯度计算	直接对参数求偏导	链式法则逐层反传
更新公式	$w := w - \alpha \frac{\partial J}{\partial w}$	相同

4.10 Python 实现——Keras 全连接网络（MNIST 手写数字识别）

Listing 14: Keras 全连接神经网络示例 (MNIST)

```

1 import numpy as np
2 from tensorflow import keras
3 from tensorflow.keras import layers
4
5 (x_train, y_train), (x_test, y_test) = keras.datasets.mnist.
    load_data()
6 x_train = x_train.reshape(-1, 784).astype('float32') / 255.0
7 x_test  = x_test.reshape(-1, 784).astype('float32') / 255.0
8 y_train = keras.utils.to_categorical(y_train, 10)
9 y_test  = keras.utils.to_categorical(y_test, 10)
10
11 model = keras.Sequential([
12     layers.Dense(128, activation='relu', input_shape=(784,)),
13     layers.Dropout(0.2),
14     layers.Dense(64, activation='relu'),
15     layers.Dropout(0.2),
16     layers.Dense(10, activation='softmax')
17 ])
18
19 model.compile(optimizer='adam',
20               loss='categorical_crossentropy',
21               metrics=['accuracy'])
22
23 history = model.fit(x_train, y_train,
24                     batch_size=128, epochs=5,
25                     validation_split=0.2, verbose=0)

```

```
26
27 test_loss, test_acc = model.evaluate(x_test, y_test, verbose=0)
28 print(f"测试准确率: {test_acc:.2%}")
29
30 pred = model.predict(x_test[:5], verbose=0)
31 print("预测类别:", np.argmax(pred, axis=1))
32 print("真实类别:", np.argmax(y_test[:5], axis=1))
```

4.11 从零实现——NumPy 版全连接网络

Listing 15: NumPy 手动实现两层神经网络

```
1 import numpy as np
2
3 def sigmoid(x):
4     return 1 / (1 + np.exp(-x))
5
6 def sigmoid_derivative(x):
7     return sigmoid(x) * (1 - sigmoid(x))
8
9 class SimpleNN:
10     def __init__(self, input_size, hidden_size, output_size):
11         self.W1 = np.random.randn(input_size, hidden_size) * 0.01
12         self.b1 = np.zeros((1, hidden_size))
13         self.W2 = np.random.randn(hidden_size, output_size) *
14             0.01
15         self.b2 = np.zeros((1, output_size))
16
17     def forward(self, X):
18         self.z1 = np.dot(X, self.W1) + self.b1
19         self.a1 = sigmoid(self.z1)
20         self.z2 = np.dot(self.a1, self.W2) + self.b2
21         self.a2 = sigmoid(self.z2)
22         return self.a2
23
24     def backward(self, X, y, lr=0.1):
25         m = X.shape[0]
26         dz2 = self.a2 - y
27         dW2 = np.dot(self.a1.T, dz2) / m
28         db2 = np.sum(dz2, axis=0, keepdims=True) / m
```

```

28         dz1 = np.dot(dz2, self.W2.T) * sigmoid_derivative(self.z1
29             )
30         dW1 = np.dot(X.T, dz1) / m
31         db1 = np.sum(dz1, axis=0, keepdims=True) / m
32         self.W2 -= lr * dW2; self.b2 -= lr * db2
33         self.W1 -= lr * dW1; self.b1 -= lr * db1
34     print("两层神经网络手动实现完成（前向+反向传播）")

```

4.12 卷积神经网络（CNN）基础——卷积、ReLU 与池化

全连接网络处理图像时会把每个像素作为独立特征，丢失了空间结构信息。卷积神经网络（CNN）通过卷积核滑动扫描图像，保留局部空间关系，是图像识别的主流架构。

4.12.1 卷积层（Convolution Layer）

卷积操作：用一个小的卷积核（滤波器）在输入特征图上滑动，每个位置计算核与覆盖区域的逐元素乘积之和。

输出尺寸公式：

$$H_{\text{out}} = \left\lfloor \frac{H_{\text{in}} + 2P - K}{S} \right\rfloor + 1, \quad W_{\text{out}} = \left\lfloor \frac{W_{\text{in}} + 2P - K}{S} \right\rfloor + 1 \quad (31)$$

其中 $H_{\text{in}}, W_{\text{in}}$ 为输入尺寸， K 为卷积核大小， P 为填充（Padding）， S 为步长（Stride）， $\lfloor \cdot \rfloor$ 表示向下取整。

手算示例：输入为 5×5 矩阵，卷积核 $K = 3 \times 3$ ，Padding $P = 1$ ，Stride $S = 1$ ：

$$X = \begin{bmatrix} 1 & 0 & 2 & 1 & 0 \\ 0 & 1 & 0 & 2 & 1 \\ 2 & 0 & 1 & 0 & 2 \\ 1 & 2 & 0 & 1 & 0 \\ 0 & 1 & 2 & 0 & 1 \end{bmatrix}, \quad K = \begin{bmatrix} 1 & 0 & -1 \\ 0 & 1 & 0 \\ -1 & 0 & 1 \end{bmatrix}$$

输出尺寸： $H_{\text{out}} = \lfloor (5 + 2 \times 1 - 3)/1 \rfloor + 1 = 5$ ，即 5×5 特征图。

以左上角位置 (0,0) 为例（Padding 后用 0 填充的边界），该位置的卷积结果为：

$$\text{output}[0,0] = (0 \times 1 + 0 \times 0 + 0 \times (-1)) + (0 \times 0 + 1 \times 1 + 0 \times 0) + (0 \times (-1) + 0 \times 0 + 2 \times 1) = 3$$

4.12.2 ReLU 激活层

卷积后通常紧跟 ReLU 激活函数，将特征图中所有负值置为 0，正值保持不变：

$$\text{ReLU}(x) = \max(0, x) \quad (32)$$

作用：引入非线性，使网络能学习复杂模式；同时稀疏激活有助于缓解过拟合。

示例：对上一步卷积结果应用 ReLU——负值变为 0, 正值不变, 如 $[-1, 3, 0, -2, 5] \xrightarrow{\text{ReLU}} [0, 3, 0, 0, 5]$ 。

4.12.3 池化层 (Pooling Layer)

池化对特征图进行下采样，降低空间维度、减少计算量、增加感受野。

最大池化 (Max Pooling)：在每个窗口内取最大值，保留最显著特征：

$$\text{MaxPool}(x)_{i,j} = \max_{m,n \in \text{window}} x_{i+m,j+n} \quad (33)$$

平均池化 (Average Pooling)：在每个窗口内取平均值，平滑整体信息：

$$\text{AvgPool}(x)_{i,j} = \frac{1}{wh} \sum_{m,n \in \text{window}} x_{i+m,j+n} \quad (34)$$

池化输出尺寸公式 (池化核 $K_p \times K_p$, 步长 S_p)：

$$H_{\text{out}} = \left\lceil \frac{H_{\text{in}}}{S_p} \right\rceil, \quad W_{\text{out}} = \left\lceil \frac{W_{\text{in}}}{S_p} \right\rceil \quad (35)$$

手算示例：输入 4×4 特征图， 2×2 最大池化，stride=2：

$$\text{输入} = \begin{bmatrix} 3 & 1 & 2 & 0 \\ 2 & 8 & 5 & 1 \\ 7 & 0 & 4 & 2 \\ 1 & 6 & 3 & 9 \end{bmatrix} \xrightarrow{\text{MaxPool } 2 \times 2, S=2} \text{输出} = \begin{bmatrix} \max(3, 1, 2, 8) = 8 & \max(2, 0, 5, 1) = 5 \\ \max(7, 0, 1, 6) = 7 & \max(4, 2, 3, 9) = 9 \end{bmatrix}$$

若改用平均池化：

$$\text{输出}_{\text{avg}} = \begin{bmatrix} \frac{3+1+2+8}{4} = 3.5 & \frac{2+0+5+1}{4} = 2 \\ \frac{7+0+1+6}{4} = 3.5 & \frac{4+2+3+9}{4} = 4.5 \end{bmatrix}$$

最大池化 vs 平均池化对比：

- **最大池化**：保留最显著特征（边缘、纹理），对微小位移具有不变性，CNN 中最常用
- **平均池化**：平滑整体信息，保留背景和整体特征分布，常用于网络末层

4.12.4 Python 实现——卷积 + ReLU + 池化

Listing 16: NumPy 手动实现卷积 +ReLU+ 最大池化

```
1 import numpy as np
2
3 def conv2d(X, K, stride=1, padding=0):
4     """二维卷积（单通道）"""
5     h_in, w_in = X.shape
6     k_h, k_w = K.shape
7     if padding > 0:
8         X_pad = np.pad(X, padding, mode='constant')
9     else:
10        X_pad = X
11    h_out = (h_in + 2 * padding - k_h) // stride + 1
12    w_out = (w_in + 2 * padding - k_w) // stride + 1
13    output = np.zeros((h_out, w_out))
14    for i in range(h_out):
15        for j in range(w_out):
16            ii, jj = i * stride, j * stride
17            region = X_pad[ii:ii+k_h, jj:jj+k_w]
18            output[i, j] = np.sum(region * K)
19    return output
20
21 def relu(x):
22     return np.maximum(0, x)
23
24 def max_pool2d(X, pool_size=2, stride=2):
25     """最大池化"""
26     h_in, w_in = X.shape
27     h_out = int(np.ceil(h_in / stride))
28     w_out = int(np.ceil(w_in / stride))
29     output = np.zeros((h_out, w_out))
30     for i in range(h_out):
31         for j in range(w_out):
32             ii, jj = i * stride, j * stride
33             region = X[ii:min(ii+pool_size, h_in),
34                        jj:min(jj+pool_size, w_in)]
35             output[i, j] = np.max(region)
36     return output
37
38 # 测试示例
39 X = np.array([[1,0,2,1,0],[0,1,0,2,1],[2,0,1,0,2],
```

```

40         [1,2,0,1,0],[0,1,2,0,1]])
41 K = np.array([[1,0,-1],[0,1,0],[-1,0,1]])
42
43 conv_out = conv2d(X, K, stride=1, padding=1)
44 relu_out = relu(conv_out)
45 pool_out = max_pool2d(relu_out, pool_size=2, stride=2)
46 print(f"卷积输出 {conv_out.shape}:\n{conv_out}")
47 print(f"ReLU输出 {relu_out.shape}:\n{relu_out}")
48 print(f"池化输出 {pool_out.shape}:\n{pool_out}")

```

典型 CNN 架构总结（从输入到输出）：

Input $\xrightarrow{\text{Conv}}$ 特征图 $\xrightarrow{\text{ReLU}}$ 激活特征 $\xrightarrow{\text{Pool}}$ 降采样特征 $\xrightarrow{\text{Flatten}}$ 一维向量 $\xrightarrow{\text{FC}}$ 分类输出

其中 Flatten 将二维特征图展平为一维向量，FC（Fully Connected）即为前面学习的全连接层，连接 CNN 特征提取与最终分类。

5 第五部分：知识表示

知识表示（Knowledge Representation）是人工智能的核心问题之一，研究如何将人类知识表示为计算机可处理的形式。本章介绍五种主要的知识表示方法：谓词逻辑、产生式规则、语义网络、框架和知识图谱。它们分别从逻辑推理、条件—动作、语义关系、结构化槽值和图谱三元组等不同角度组织知识。

5.1 谓词逻辑表示法

谓词逻辑是 AI 中最经典的知识表示方法之一，用谓词和量词描述事实和规则。

一阶谓词逻辑的语法要素：

- 个体词：表示对象（常量、变量）
- 谓词：表示对象的属性或关系，如 $\text{Father}(x, y)$, $\text{Greater}(x, 5)$
- 量词： $\forall$ （全称量词）、 $\exists$ （存在量词）
- 联结词： $\wedge$ （合取）、 $\vee$ （析取）、 $\neg$ （否定）、 $\rightarrow$ （蕴含）

示例：用谓词逻辑表示「所有人都会死，苏格拉底是人，所以苏格拉底会死」：

$$\forall x [\text{Human}(x) \rightarrow \text{Mortal}(x)] \quad (36)$$

$$\text{Human}(\text{Socrates}) \quad (37)$$

$$\therefore \text{Mortal}(\text{Socrates}) \quad (38)$$

5.1.1 Python 实现——谓词逻辑归结推理演示

归结（Resolution）是谓词逻辑中核心的自动推理规则：若已知 $A \vee B$ 和 $\neg A \vee C$ ，则可推出 $B \vee C$ 。以下是基于 Python 的谓词归结推理实现：

Listing 17: 谓词逻辑归结推理——命题化演示

```

1 def resolve(clause1, clause2):
2     """对两个子句进行归结推理"""
3     resolvents = set()
4     for lit in clause1:
5         complement = '-' + lit if not lit.startswith('-') else
6             lit[1:]
7         if complement in clause2:
8             new_clause = (clause1 - {lit}) | (clause2 - {
9                 complement})
10            if not new_clause:
11                return {'EMPTY'} # 空子句 => 矛盾，证明成功
12            resolvents.add(frozenset(new_clause))
13    return resolvents
14
15 def resolution_proof(kb, goal_negation):
16     """归结证明：从知识库和否定目标推出空子句"""
17     clauses = set(kb) | {frozenset(goal_negation)}
18     new_clauses = set()
19     while True:
20         pairs = [(c1,c2) for c1 in clauses for c2 in clauses if
21             c1 < c2]
22         for c1, c2 in pairs:
23             res = resolve(c1, c2)
24             if 'EMPTY' in res:
25                 return True # 归结成功
26             new_clauses |= res
27         if new_clauses.issubset(clauses):
28             return False # 无法推出新子句
29         clauses |= new_clauses
30
31 # 示例1：证明苏格拉底会死
32 kb1 = [
33     frozenset({'-Human', 'Mortal'}), # ~Human OR Mortal

```

```

33     frozenset({'Human'}),                                # Human(Socrates)
34 ]
35 goal_neg1 = {'-Mortal'}
36 print(f"苏格拉底证明: {'成功' if resolution_proof(kb1, goal_neg1)
37     else '失败'}")
37
38 # 示例2: 亲属关系推理
39 kb2 = [
40     frozenset({'-Father_x_y', 'Parent_x_y'}),
41     frozenset({'-Parent_x_y', '-Parent_y_z', 'Grandparent_x_z'}),
42     frozenset({'Father_John_Mary'}),
43     frozenset({'Father_Mary_Tom'}),
44 ]
45 goal_neg2 = {'-Grandparent_John_Tom'}
46 print(f"亲属关系推理: {'推理成功' if resolution_proof(kb2,
47     goal_neg2) else '推理失败'}")
47
48 # 示例3: 电路故障诊断
49 kb3 = [
50     frozenset({'-OK_Battery', '-OK_Starter', 'EngineStarts'}),
51     frozenset({'OK_Battery'}),
52     frozenset({'OK_Starter'}),
53 ]
54 goal_neg3 = {'-EngineStarts'}
55 print(f"电路诊断: {'引擎会启动' if resolution_proof(kb3,
56     goal_neg3) else '引擎不会启动'}")

```

5.2 产生式表示法

产生式规则是「IF-THEN」形式的条件—动作对，格式为：

IF < 条件 > THEN < 动作/结论 >

示例：动物识别系统

IF 有毛发 THEN 是哺乳动物 (39)

IF 是哺乳动物 $\wedge$ 有蹄 THEN 是有蹄类动物 (40)

IF 是有蹄类动物 $\wedge$ 有黑色条纹 THEN 是斑马 (41)

Python 实现产生式推理：

Listing 18: 产生式规则示例——正向推理框架

```

1 rules = [
2     {"if": {"有毛发"},          "then": "哺乳动物"},
3     {"if": {"哺乳动物", "有蹄"}, "then": "有蹄类"},
4     {"if": {"有蹄类", "黑色条纹"}, "then": "斑马"},
5     {"if": {"有蹄类", "长脖子"},  "then": "长颈鹿"},
6 ]
7
8 def forward_chain(facts, rules):
9     known = set(facts)
10    changed = True
11    while changed:
12        changed = False
13        for rule in rules:
14            if rule["if"].issubset(known) and rule["then"] not in
15                known:
16                known.add(rule["then"])
17                changed = True
18    return known
19
20 result = forward_chain({"有毛发", "有蹄", "黑色条纹"}, rules)
21 print(f"推理结果: {result}")

```

5.3 语义网络表示法

语义网络是用节点（实体）和有向弧（语义关系）表示知识的有向图。基本单元为三元组：(节点₁, 关系, 节点₂)。

常见关系类型：

- ISA（是一个）：实例关系，如鸵鸟 $\xrightarrow{\text{ISA}}$ 鸟
- AKO（是一种）：分类关系，如鸟 $\xrightarrow{\text{AKO}}$ 动物
- A-Member-Of：成员关系，如张强 $\xrightarrow{\text{A-Member-Of}}$ 共青团员
- 属性关系：Have, Can, Located 等

5.3.1 Python 实现——语义网络构建与查询

Listing 19: 语义网络构建与继承推理

```

1 from collections import deque

```

```
2
3 class SemanticNetwork:
4     """语义网络：节点和有向关系弧构成的图结构"""
5     def __init__(self):
6         self.edges = {} # (node, relation) -> [target_nodes]
7
8     def add_relation(self, node1, relation, node2):
9         key = (node1, relation)
10        if key not in self.edges:
11            self.edges[key] = []
12        self.edges[key].append(node2)
13
14    def query(self, node, relation):
15        return self.edges.get((node, relation), [])
16
17    def trace_inheritance(self, node):
18        """沿ISA/AKO链向上追溯继承层次"""
19        chain = [node]
20        current = node
21        while True:
22            parents = (self.query(current, 'ISA') +
23                      self.query(current, 'AKO'))
24            if not parents:
25                break
26            current = parents[0]
27            chain.append(current)
28        return chain
29
30
31 # 构建动物分类语义网络
32 net = SemanticNetwork()
33 net.add_relation('鸵鸟', 'ISA', '鸟')
34 net.add_relation('麻雀', 'ISA', '鸟')
35 net.add_relation('老虎', 'ISA', '哺乳动物')
36 net.add_relation('张强', 'ISA', '学生')
37 net.add_relation('鸟', 'AKO', '动物')
38 net.add_relation('哺乳动物', 'AKO', '动物')
39 net.add_relation('学生', 'AKO', '人')
40 net.add_relation('人', 'AKO', '动物')
41 net.add_relation('鸟', 'Can', '飞行')
```

```

42 net.add_relation('鸟', 'Have', '羽毛')
43 net.add_relation('老虎', 'Have', '条纹')
44
45 print("鸵鸟的继承链:", ' -> '.join(net.trace_inheritance('鸵鸟'))
46     )
47 print("张强的继承链:", ' -> '.join(net.trace_inheritance('张强'))
48     )
49
50 def is_a(net, instance, category):
51     return category in net.trace_inheritance(instance)
52
53 print(f"鸵鸟是动物吗? {is_a(net, '鸵鸟', '动物')}")
54 print(f"张强是动物吗? {is_a(net, '张强', '动物')}")

```

5.4 框架表示法

框架（Frame）是一种结构化的知识表示方法，由 Minsky 于 1975 年提出。每个框架由若干槽（Slot）组成，槽用于描述对象的属性、状态、关系或行为。

框架的核心要素：

- **槽（Slot）**：对象的属性或关系，如「名称」「类型」「容纳人数」
- **侧面（Facet）**：每个槽可有多个侧面，用于指定槽值类型、默认值、约束条件、求值过程等
- **继承（Inheritance）**：子框架可以从父框架继承槽结构，支持「AKO」（是一种）和「Instance」（实例）两级继承

框架的一般结构：

$$\text{Frame} = \{\text{Slot}_1 : \text{value}_1; \text{Slot}_2 : \text{value}_2; \dots\}$$

框架与语义网络的关系：框架可以看作语义网络的节点结构化扩展——语义网络的节点用弧连接，框架用槽填充。框架更适合描述具有固定结构的对象（如房间、人物档案），语义网络更适合描述灵活的关系网络。

示例——教室框架的槽—侧面结构：

Listing 20: 框架表示法 Python 实现

```

1 class Frame:
2     def __init__(self, name):
3         self.name = name
4         self.slots = {}

```

表 4: 「教室」框架的槽—侧面定义

槽名	值类型	默认值	约束
名称	字符串	N/A	非空
类型	枚举	普通教室	{普通, 多媒体, 实验室}
容纳人数	整数	30	> 0
是否有投影仪	布尔	False	—
所在楼层	整数	1	1—6

```

5
6     def add_slot(self, slot_name, value=None, default=None,
7                   constraint=None, if_needed=None):
8         self.slots[slot_name] = {
9             "value": value, "default": default,
10            "constraint": constraint, "if_needed": if_needed
11        }
12
13 classroom = Frame("教室")
14 classroom.add_slot("名称", value="N301")
15 classroom.add_slot("类型", value="多媒体教室")
16 classroom.add_slot("容纳人数", value=60, constraint="int>0")
17 classroom.add_slot("是否有投影仪", value=True)
18 classroom.add_slot("所在楼层", value=3)
19
20 for slot, info in classroom.slots.items():
21     print(f" {slot}: {info['value']}")

```

5.5 知识图谱

知识图谱 (Knowledge Graph, KG) 是 Google 于 2012 年正式提出的概念, 以三元组 (头实体, 关系, 尾实体) 的形式组织知识, 可看作大规模语义网络的现代工程化实现。

知识图谱的核心特征:

- **实体 (Entity):** 现实世界中的对象, 如人物、地点、机构、概念等
- **关系 (Relation):** 实体间的语义联系, 如「父亲」「位于」「创立于」
- **三元组:** (头实体, 关系, 尾实体), 如 (贾宝玉, 父亲, 贾政)

知识图谱的主要类型:

- **通用知识图谱**：覆盖广泛领域，如 Google Knowledge Graph、Wikidata、DBpedia
 - **领域知识图谱**：聚焦特定行业，如医疗知识图谱、金融知识图谱、法律知识图谱
- 与语义网络的关键区别：

- 语义网络是**知识表示模型**（理论层面），节点和弧的语义可以自由定义
- 知识图谱是**知识工程系统**（工程层面），强调大规模存储、检索、推理和可视化
- 知识图谱通常建立在图数据库（如 Neo4j）之上，支持 SPARQL/Cypher 等图查询语言

知识图谱的典型应用：搜索引擎（Google 知识面板）、智能问答（Siri/小爱）、推荐系统（关联推荐）、反欺诈（关系网络分析）。

5.5.1 Python 实现——知识图谱构建与查询

Listing 21: 知识图谱构建与关系路径查询

```
1 from collections import deque
2
3 class KnowledgeGraph:
4     """简易知识图谱：三元组存储与图遍历查询"""
5     def __init__(self):
6         self.triples = []
7
8     def add(self, head, relation, tail):
9         self.triples.append((head, relation, tail))
10
11    def query_head(self, head, relation=None):
12        return [(h,r,t) for h,r,t in self.triples
13                if h == head and (relation is None or r ==
14                                relation)]
15
16    def query_tail(self, tail, relation=None):
17        return [(h,r,t) for h,r,t in self.triples
18                if t == tail and (relation is None or r ==
19                                relation)]
20
21    def find_path(self, start, end, max_depth=4):
22        """BFS查找两个实体间的关系路径"""
23        if start == end:
```

```

22         return [[start]]
23     queue = deque([(start, [start])])
24     visited = {start}
25     while queue:
26         current, path = queue.popleft()
27         if len(path) > max_depth:
28             continue
29         for h, r, t in self.triples:
30             if h == current and t not in visited:
31                 new_path = path + [r, t]
32                 if t == end:
33                     return new_path
34                 visited.add(t)
35                 queue.append((t, new_path))
36     return None
37
38
39 # 构建红楼梦人物关系知识图谱
40 kg = KnowledgeGraph()
41 kg.add('贾宝玉', '父亲', '贾政')
42 kg.add('贾宝玉', '母亲', '王夫人')
43 kg.add('贾宝玉', '祖母', '贾母')
44 kg.add('贾政', '母亲', '贾母')
45 kg.add('林黛玉', '父亲', '林如海')
46 kg.add('林黛玉', '外祖母', '贾母')
47 kg.add('薛宝钗', '母亲', '薛姨妈')
48 kg.add('王夫人', '姐妹', '薛姨妈')
49 kg.add('贾宝玉', '表姐', '薛宝钗')
50 kg.add('贾宝玉', '表妹', '林黛玉')
51 kg.add('贾宝玉', '居住', '怡红院')
52 kg.add('林黛玉', '居住', '潇湘馆')
53
54 print("=== 贾宝玉的直接关系 ===")
55 for h, r, t in kg.query_head('贾宝玉'):
56     print(f" {h} --{r}--> {t}")
57
58 print("\n=== 关系路径查询 ===")
59 path = kg.find_path('林黛玉', '薛宝钗')
60 if path:
61     print(' -> '.join(path))

```

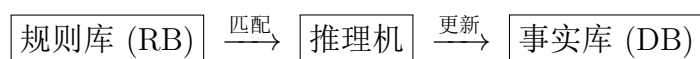
6 第六部分：推理方法

6.1 产生式推理

6.1.1 产生式系统的三大组成部分

产生式系统（Production System）由三部分组成：

- **规则库（Rule Base, RB）**：存储所有产生式规则，每条规则为「IF 条件 THEN 结论」形式。规则之间相互独立，便于增删修改。
- **事实库/综合数据库（Working Memory / Database, DB）**：存放当前已知的事实和中间推理结果，是动态变化的。
- **控制系统/推理机（Control System / Inference Engine）**：负责选择和执行规则，包含**匹配**（找出条件满足的规则）、**冲突消解**（从多条可用规则中选一条）、**执行**（将结论加入事实库）三步循环。



6.1.2 冲突消解策略（Conflict Resolution）

当多条规则的条件同时被满足时，推理机必须决定优先执行哪条规则。常见策略：

- **优先级排序**：为每条规则预设优先级，选优先级最高的
- **特殊性优先**：条件更多的规则更「具体」，优先执行
- **最近使用优先**：优先采用最新加入事实库的规则
- **顺序优先**：按规则在规则库中的排列顺序依次尝试

6.1.3 正向推理（数据驱动）

从已知事实出发，反复匹配规则的前提，推出新事实，直到达到目标或无可匹配规则。正向推理适合**监测、诊断**等场景——从观察到的现象出发推断结论。

6.1.4 逆向推理（目标驱动）

从目标出发，反向寻找能推出目标的规则，将其前提作为新的子目标递归求解。逆向推理适合**验证假设**——先猜测一个结论，再寻找支持证据。

正向推理 vs 逆向推理对比：

表 5: 正向推理与逆向推理对比

维度	正向推理	逆向推理
驱动方式	数据驱动	目标驱动
推理方向	事实 $\rightarrow$ 结论	目标 $\rightarrow$ 前提（子目标）
优点	可发现意外结论	目标明确，效率高
缺点	可能产生大量无关结论	需要已知目标
适用场景	监测、诊断、模式发现	验证假设、定理证明

6.1.5 逆向推理 Python 实现

Listing 22: 逆向推理——目标驱动的规则匹配

```

1 def backward_chain(goal, facts, rules, visited=None):
2     """逆向推理：从目标出发反向查找前提"""
3     if visited is None:
4         visited = set()
5     if goal in visited:
6         return False # 防止循环推理
7     visited.add(goal)
8
9     if goal in facts:
10        return True # 目标已是已知事实
11
12    for conds, concl in rules:
13        if concl == goal:
14            # 该规则能推出目标，递归验证所有前提
15            if all(backward_chain(c, facts, rules, visited.copy())
16                  ):
17                for c in conds:
18                    print(f"规则触发: {conds} -> {concl}")
19                return True
20    return False
21
22 # 示例
23 facts = {'有毛发', '有蹄', '黑色条纹'}
24 rules = [
25     ({'有毛发'}, '哺乳动物'),
26     ({'哺乳动物', '有蹄'}, '有蹄类'),
27     ({'有蹄类', '黑色条纹'}, '斑马'),

```

```

27 ]
28 result = backward_chain('斑马', facts, rules)
29 print(f'推理结果: {"斑马"} 成立' if result else '推理失败')

```

6.1.6 正向推理 Python 实现

Listing 23: 正向推理完整实现

```

1 class ProductionSystem:
2     def __init__(self):
3         self.facts = set()
4         self.rules = [] # (条件集合, 结论)
5
6     def add_fact(self, fact):
7         self.facts.add(fact)
8
9     def add_rule(self, conditions, conclusion):
10        self.rules.append((set(conditions), conclusion))
11
12    def forward_reasoning(self, goal=None):
13        changed = True
14        trace = []
15        while changed:
16            changed = False
17            for conds, concl in self.rules:
18                if conds.issubset(self.facts) and concl not in
19                    self.facts:
20                    self.facts.add(concl)
21                    trace.append(f"规则: {conds} -> {concl}")
22                    changed = True
23                    if goal and concl == goal:
24                        return True, trace
25            return (goal in self.facts if goal else True), trace
26
27 ps = ProductionSystem()
28 ps.add_fact("有毛发"); ps.add_fact("产奶")
29 ps.add_rule({"有毛发"}, "哺乳动物")
30 ps.add_rule({"产奶"}, "哺乳动物")
31 ps.add_rule({"哺乳动物", "吃肉"}, "食肉动物")

```

```

32 success, trace = ps.forward_reasoning("哺乳动物")
33 print("推理成功:", success)
34 for t in trace: print(t)

```

6.2 可信度推理 (Certainty Factor)

6.2.1 可信度模型

可信度 CF (Certainty Factor) 是人们对事物为真程度的判断, 取值范围 $[-1, 1]$:

- $CF > 0$: 证据支持结论, 越大支持度越高
- $CF < 0$: 证据反对结论, 越小反对度越高
- $CF = 0$: 证据与结论无关

知识表示为: **IF** E **THEN** H ($CF(H, E)$)。

6.2.2 CF 的计算公式

信任增长度 MB 和不信任增长度 MD :

$$CF(H, E) = MB(H, E) - MD(H, E) \quad (42)$$

证据不确定性传递 (证据 E 的可信度为 $CF(E)$):

$$CF(H) = CF(H, E) \cdot \max\{0, CF(E)\} \quad (43)$$

合取与析取证据:

$$CF(E_1 \wedge E_2) = \min\{CF(E_1), CF(E_2)\} \quad (44)$$

$$CF(E_1 \vee E_2) = \max\{CF(E_1), CF(E_2)\} \quad (45)$$

结论合成公式 (多条规则推出同一结论 H): 设规则 1 推出 H 的可信度为 $CF_1(H)$, 规则 2 推出同一 H 的可信度为 $CF_2(H)$, 则合成后的可信度 $CF_{1,2}(H)$ 为:

$$CF_{1,2}(H) = \begin{cases} CF_1(H) + CF_2(H) - CF_1(H) \cdot CF_2(H), & CF_1(H) \geq 0, CF_2(H) \geq 0 \\ CF_1(H) + CF_2(H) + CF_1(H) \cdot CF_2(H), & CF_1(H) < 0, CF_2(H) < 0 \\ \frac{CF_1(H) + CF_2(H)}{1 - \min\{|CF_1(H)|, |CF_2(H)|\}}, & \text{否则 (异号)} \end{cases} \quad (46)$$

其中 $CF_1(H) = CF(H, E_1) \cdot \max\{0, CF(E_1)\}$ 为第一条规则传递后的可信度, $CF_2(H)$ 同理。

6.2.3 CF 推理完整算例

已知规则：

$$r_1 : \text{IF } E_1 \text{ THEN } H \quad (CF(H, E_1) = 0.9)$$

$$r_2 : \text{IF } E_2 \text{ THEN } H \quad (CF(H, E_2) = 0.6)$$

$$r_3 : \text{IF } E_3 \text{ AND } (E_4 \text{ OR } E_5) \text{ THEN } E_1 \quad (CF(E_1, \cdot) = 0.8)$$

已知初始证据可信度： $CF(E_2) = 0.9$, $CF(E_3) = 0.8$, $CF(E_4) = 0.5$, $CF(E_5) = 0.6$
求 $CF(H)$ 。

步骤一：计算 $CF(E_1)$

先计算 r_3 的前提（组合证据）可信度：

$$CF(E_4 \vee E_5) = \max\{CF(E_4), CF(E_5)\} = \max\{0.5, 0.6\} = 0.6 \quad (47)$$

$$CF(E_3 \wedge (E_4 \vee E_5)) = \min\{CF(E_3), CF(E_4 \vee E_5)\} = \min\{0.8, 0.6\} = 0.6 \quad (48)$$

然后通过 r_3 传递得到 E_1 的可信度：

$$CF(E_1) = CF(E_1, \cdot) \cdot \max\{0, 0.6\} = 0.8 \times 0.6 = 0.48$$

步骤二：分别计算 r_1 和 r_2 传递给 H 的可信度

$$CF_1(H) = CF(H, E_1) \cdot \max\{0, CF(E_1)\} = 0.9 \times 0.48 = 0.432 \quad (49)$$

$$CF_2(H) = CF(H, E_2) \cdot \max\{0, CF(E_2)\} = 0.6 \times 0.9 = 0.540 \quad (50)$$

步骤三：合成两个可信度

$$CF_{1,2}(H) = CF_1(H) + CF_2(H) - CF_1(H) \cdot CF_2(H) = 0.432 + 0.540 - 0.2333 = 0.7387$$

最终结论： $CF(H) \approx 0.74$ ，即有约 74% 的把握认为结论 H 成立。

6.2.4 Python 实现可信度推理

Listing 24: 可信度推理 CF 模型实现

```

1 def combine_cf(cf1, cf2):
2     """合成两个可信度值"""
3     if cf1 >= 0 and cf2 >= 0:
4         return cf1 + cf2 - cf1 * cf2
5     elif cf1 < 0 and cf2 < 0:
6         return cf1 + cf2 + cf1 * cf2
7     else:
8         denom = 1 - min(abs(cf1), abs(cf2))

```

```

9         return (cf1 + cf2) / denom if denom != 0 else 0
10
11 def cf_and(cf_list):
12     return min(cf_list) if cf_list else 0
13
14 def cf_or(cf_list):
15     return max(cf_list) if cf_list else 0
16
17 def propagate_cf(cf_he, cf_e):
18     return cf_he * max(0, cf_e)
19
20 # 示例
21 cf_rule1 = propagate_cf(0.8, 0.6)
22 cf_rule2 = propagate_cf(0.5, 0.9)
23 cf_final = combine_cf(cf_rule1, cf_rule2)
24 print(f"合成可信度: {cf_final:.4f}")

```

7 第七部分：搜索策略

7.1 状态空间问题求解

状态空间由三个要素组成：

$$(S, F, G)$$

其中 S 为初始状态集合， F 为操作算子集合， G 为目标状态集合。问题求解即寻找从 $S_0 \in S$ 到 $g \in G$ 的操作序列。

7.1.1 经典示例——二阶梵塔问题

梵塔问题（Tower of Hanoi）是状态空间搜索的经典问题。设有三根柱子（编号 1、2、3），初始状态下 1 号柱上有 A、B 两个金片（A 在 B 上面，A 比 B 小）。要求将两个金片全部移到另一根柱子上，每次只能移动一个金片，且任何时刻都不能使大金片位于小金片上方。

状态表示：用 (S_A, S_B) 表示金片 A 和 B 所在的柱子号。全部 9 种状态如下：

$$S_0 = (1, 1), S_1 = (1, 2), S_2 = (1, 3), S_3 = (2, 1), S_4 = (2, 2), \\ S_5 = (2, 3), S_6 = (3, 1), S_7 = (3, 2), S_8 = (3, 3)$$

操作（算符）： A_{ij} 表示将金片 A 从柱子 i 移到柱子 j ， B_{ij} 同理。共 12 种操作： $A_{12}, A_{13}, A_{21}, A_{23}, A_{31}, A_{32}, B_{12}, B_{13}, B_{21}, B_{23}, B_{31}, B_{32}$

目标状态集合： $G = \{S_4 = (2, 2), S_8 = (3, 3)\}$

初始状态 $(1, 1)$ 可通过操作序列 A_{13}, B_{12}, A_{32} 到达目标 $(3, 3)$ 。初始状态 $(1, 1)$ 可通过操作序列 A_{13}, B_{12}, A_{32} 到达目标 $(3, 3)$ 。这就是一个解。

7.1.2 盲目搜索：广度优先与深度优先

在讨论启发式搜索（A*）之前，需要先理解两种最基本的盲目搜索（无信息搜索）策略：

- **广度优先搜索（BFS）**：逐层扩展节点，先扩展根节点的所有子节点，再扩展下一层。使用队列（FIFO）实现。
 - 优点：当每步代价相等时，保证找到最短路径（完备且最优）
 - 缺点：空间复杂度高，需存储所有已生成但未扩展的节点： $O(b^d)$
- **深度优先搜索（DFS）**：沿着一条路径一直向下探索，直到无法继续再回溯。使用栈（LIFO）实现。
 - 优点：空间效率高，只需 $O(bm)$ 存储当前路径
 - 缺点：不保证找到最优解，可能陷入无限深分支（需设置深度限制）

BFS 与 DFS 对比：

表 6: BFS vs DFS 对比

维度	BFS（广度优先）	DFS（深度优先）
数据结构	队列（Queue, FIFO）	栈（Stack, LIFO）
完备性	完备（有限分支因子下）	不完备（可能无限循环）
最优性	最优（代价相等时）	不一定最优
时间复杂度	$O(b^d)$	$O(b^m)$, m 可能远大于 d
空间复杂度	$O(b^d)$ （需存储整层）	$O(bm)$ （仅存储当前路径）
适用场景	最短路径、状态空间较小	解很深、内存受限、有深度限制

一致代价搜索（UCS）：当每步代价不同时，BFS 不再保证最优。UCS 使用优先队列，每次扩展当前路径代价 $g(n)$ 最小的节点。当 $g(n)$ 为深度时，UCS 退化为 BFS。

7.1.3 Python 实现——BFS 求解八数码问题

Listing 25: BFS 求解八数码问题

```
1 from collections import deque
2
```

```

3 class EightPuzzle:
4     def __init__(self, start, goal=[1,2,3,4,5,6,7,8,0]):
5         self.start = tuple(start)
6         self.goal = tuple(goal)
7         self.moves = {0: [1,3], 1:[0,2,4], 2:[1,5],
8                         3:[0,4,6], 4:[1,3,5,7], 5:[2,4,8],
9                         6:[3,7], 7:[4,6,8], 8:[5,7]}
10
11     def bfs(self):
12         if self.start == self.goal:
13             return []
14         queue = deque([(self.start, [])])
15         visited = {self.start}
16         while queue:
17             state, path = queue.popleft()
18             zero = state.index(0)
19             for nxt in self.moves[zero]:
20                 lst = list(state)
21                 lst[zero], lst[nxt] = lst[nxt], lst[zero]
22                 new_state = tuple(lst)
23                 if new_state == self.goal:
24                     return path + [nxt]
25                 if new_state not in visited:
26                     visited.add(new_state)
27                     queue.append((new_state, path + [nxt]))
28         return None
29
30 puzzle = EightPuzzle([2,8,3,1,6,4,7,0,5])
31 result = puzzle.bfs()
32 print(f"找到解: {len(result)}步" if result else "无解")

```

7.2 启发式搜索——A* 算法

7.2.1 估价函数

$$f(n) = g(n) + h(n) \quad (51)$$

其中：

- $g(n)$: 从初始节点到节点 n 的实际代价

- $h(n)$: 从节点 n 到目标节点的启发式估计代价（启发函数）

当 $h(n)$ 满足可容许条件（不高估实际代价）时，A* 保证找到最优解。

7.2.2 A* 算法的 Python 实现

Listing 26: A* 搜索算法实现

```

1 import heapq
2
3 def a_star(start, goal, neighbors, heuristic):
4     """通用A*算法框架
5     neighbors(state) -> [(next_state, cost), ...]
6     heuristic(state) -> float
7     """
8     open_set = [(heuristic(start), 0, start, [start])]
9     g_score = {start: 0}
10
11     while open_set:
12         f, g, current, path = heapq.heappop(open_set)
13         if current == goal:
14             return path, g
15         for neighbor, cost in neighbors(current):
16             tentative_g = g + cost
17             if neighbor not in g_score or tentative_g < g_score[
18                 neighbor]:
19                 g_score[neighbor] = tentative_g
20                 f_score = tentative_g + heuristic(neighbor)
21                 heapq.heappush(open_set, (f_score, tentative_g,
22                     neighbor, path + [neighbor]))
23         return None, float('inf')
24
25 # 示例：网格路径规划
26 def grid_neighbors(pos):
27     x, y = pos
28     return [(x+dx, y+dy), 1) for dx,dy in [(1,0),(-1,0),(0,1),
29         (0,-1)]
30         if 0 <= x+dx < 10 and 0 <= y+dy < 10]
31
32 def manhattan_heuristic(pos):
33     return abs(pos[0]-9) + abs(pos[1]-9)

```

```

32 path, cost = a_star((0,0), (9,9), grid_neighbors,
    manhattan_heuristic)
33 print(f"A*路径长度: {cost}, 步数: {len(path)}")

```

7.3 遗传算法 (Genetic Algorithm)

7.3.1 基本原理

遗传算法模拟自然进化过程寻找最优解，核心操作包括：

- **编码**：将解表示为染色体（如二进制串或实数向量）
- **选择 (Selection)**：轮盘赌选择，适应度越高被选中概率越大
- **交叉 (Crossover)**：交换两个父代的基因片段生成子代
- **变异 (Mutation)**：以概率 P_m 随机改变基因值

选择概率（轮盘赌）：

$$P(\text{select}_i) = \frac{f_i}{\sum_{j=1}^N f_j} \quad (52)$$

7.3.2 Python 实现

Listing 27: 遗传算法求解函数最大值

```

1  import numpy as np
2
3  def genetic_algorithm(fitness_fn, pop_size=50, gens=100,
4                        crossover_rate=0.8, mutation_rate=0.01):
5      pop = np.random.randint(0, 2, (pop_size, 10))
6
7      for gen in range(gens):
8          values = np.array([int(''.join(map(str, ind)), 2)
9                             for ind in pop])
10         fitness = np.array([fitness_fn(v) for v in values])
11         fitness = fitness - fitness.min() + 1e-6
12
13         prob = fitness / fitness.sum()
14         parents_idx = np.random.choice(pop_size, size=pop_size, p
15                                         =prob)
16         parents = pop[parents_idx]

```

```
17     offspring = parents.copy()
18     for i in range(0, pop_size-1, 2):
19         if np.random.rand() < crossover_rate:
20             point = np.random.randint(1, 9)
21             offspring[i, point:], offspring[i+1, point:] = \
22                 parents[i+1, point:].copy(), parents[i, point
23                     :].copy()
24
25     mask = np.random.rand(pop_size, 10) < mutation_rate
26     offspring = np.where(mask, 1 - offspring, offspring)
27     pop = offspring
28
29     best_idx = np.argmax([fitness_fn(int(''.join(map(str, ind)), 2)
30                             )
31                             for ind in pop])
32     return int(''.join(map(str, pop[best_idx])), 2)
33
34 # 求解  $f(x)=x*\sin(x)$  在  $[0,1023]$  的最大值
35 best_x = genetic_algorithm(lambda x: x * np.sin(x/100))
36 print(f"最优解 x={best_x}, f(x)={best_x*np.sin(best_x/100):.4f}")
```

8 附录：课后补充练习题要点

课后练习题覆盖以下核心知识点：

- **AI 基本概念：** AI 定义、三大学派特点与区别、强 AI 与弱 AI 区分
- **机器学习基础：** 监督/无监督学习区别、回归与分类区分
- **Python 编程：** 数据类型判断、列表与字典操作、NumPy 数组操作
- **线性回归：** 模型形式 $f(\mathbf{x}) = \mathbf{w}^T \mathbf{x} + b$ 、最小二乘法、梯度下降法
- **逻辑斯蒂分类：** Sigmoid 函数、Accuracy/Precision/Recall/F1
- **K-近邻：** KNN 原理、距离度量 (L_1, L_2)、K 值选择
- **K-均值：** 聚类目标函数、质心更新、肘部法则
- **朴素贝叶斯：** 贝叶斯公式、离散型 (MultinomialNB+ 拉普拉斯平滑) 与连续型 (GaussianNB) 的区别
- **知识表示：** 谓词逻辑、产生式规则、语义网络 (ISA/AKO)、框架表示、知识图谱
- **推理方法：** 正向/逆向推理流程、CF 模型计算公式
- **搜索策略：** 状态空间三要素 (S, F, G)、A* 估价函数 $f(n) = g(n) + h(n)$ 、遗传算法流程
- **神经网络：** 全连接结构、前向传播公式、激活函数 (Sigmoid/ReLU/Softmax)、反向传播

8.1 关键公式汇总

表 7: 课程核心公式一览表

算法	核心公式
线性回归	$f(\mathbf{x}) = \mathbf{w}^T \mathbf{x} + b$
MSE 损失	$J = \frac{1}{2m} \sum (f(\mathbf{x}^{(i)}) - y^{(i)})^2$
R^2 决定系数	$R^2 = 1 - \frac{\sum (y^{(i)} - \hat{y}^{(i)})^2}{\sum (y^{(i)} - \bar{y})^2}$
调整 R^2	$R_{\text{adj}}^2 = 1 - \frac{(1-R^2)(m-1)}{m-d-1}$
Sigmoid 函数	$\sigma(z) = \frac{1}{1+e^{-z}}$
交叉熵损失	$J = -\frac{1}{m} \sum [y \ln \hat{y} + (1-y) \ln(1-\hat{y})]$
Accuracy	$\text{Acc} = \frac{TP+TN}{TP+TN+FP+FN}$
Precision	$P = \frac{TP}{TP+FP}$
Recall	$R = \frac{TP}{TP+FN}$
F1 分数	$F_1 = \frac{2PR}{P+R}$
欧氏距离	$d = \sqrt{\sum (x_i - y_i)^2}$
贝叶斯公式	$P(c_k \mathbf{x}) \propto P(c_k) \prod P(x_i c_k)$
拉普拉斯平滑	$P(x_i = v c_k) = \frac{\text{count}(x_i=v, c_k) + \lambda}{\text{count}(c_k) + \lambda V_i }$
高斯贝叶斯	$P(x_i c_k) = \frac{1}{\sqrt{2\pi}\sigma^2} e^{-\frac{(x_i - \mu)^2}{2\sigma^2}}$
前向传播	$\hat{y} = f(\mathbf{W}_2 \cdot g(\mathbf{W}_1 \mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2)$
CF 模型	$CF(H) = CF(H, E) \cdot \max\{0, CF(E)\}$
偏差-方差	$\text{Error} = \text{Bias}^2 + \text{Variance} + \sigma^2$
A* 估价函数	$f(n) = g(n) + h(n)$